



You Do It

Using Object References

Next, you write an application in which you create an array of `Vehicle` references. Within the application, you assign `Sailboat` objects and `Bicycle` objects to the same array. Then, because the different object types are stored in the same array, you can easily manipulate them by using a `for` loop.

1. Open a new file, and then enter the following first few lines of the `VehicleDatabase` program:
2. Create the following array of five `Vehicle` references and an integer subscript to use with the array:

```
Vehicle[] vehicles = new Vehicle[5];
int x;
```

3. Enter the following `for` loop that prompts you to select whether to enter a sailboat or a bicycle in the array. Based on user input, instantiate the appropriate object type.

```
for(x = 0; x < vehicles.length; ++x)
{
    String userEntry;
    int vehicleType;
    userEntry = JOptionPane.showInputDialog(null,
        "Please select the type of\n " +
        "vehicle you want to enter: \n1 - Sailboat\n" +
        " 2 - Bicycle");
    vehicleType = Integer.parseInt(userEntry);
    if(vehicleType == 1)
        vehicles[x] = new Sailboat();
    else
        vehicles[x] = new Bicycle();
}
```

(continues)

(continued)

4. After entering the information for each vehicle, display the array contents by typing the following code. First create a `StringBuffer` to hold the list of vehicles. Then, in a `for` loop, build an output `String` by repeatedly adding a newline character, a counter, and a vehicle from the array to the `StringBuffer` object. Display the constructed `StringBuffer` in a dialog box. Then type the closing curly braces for the `main()` method and the class:

```
StringBuffer outString = new StringBuffer();
for(x = 0; x < vehicles.length; ++x)
{
    outString.append("\n#" + (x + 1) + " ");
    outString.append(vehicles[x].toString());
}
OptionPane.showMessageDialog(null,
    "Our available Vehicles include:\n" +
    outString);
}
```

5. Save the file as **VehicleDatabase.java**, and then compile it. Run the application, entering five objects of your choice. Figure 11-14 shows typical output after the user has entered data.

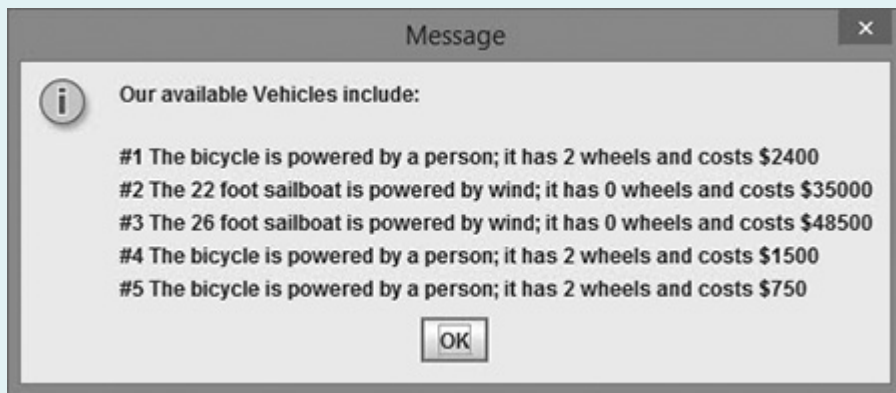


Figure 11-14 Typical output of the `VehicleDatabase` application

Using the Object Class and Its Methods

Every class in Java is actually a subclass, except one. When you define a class, if you do not explicitly extend another class, your class implicitly is an extension of the `Object` class. The **Object class** is defined in the `java.lang` package, which is imported automatically every